

IF25-22017

PENGEMBANGAN APLIKASI MOBILE

PERTEMUAN 12

Project Sprint 2: Core Features

UI Implementation, Navigation, dan Data Layer

Program Studi Teknik Informatika
Institut Teknologi Sumatera
Tahun Akademik Genap 2025/2026



SPRINT PROGRESS

W11	Sprint 1: Planning & Setup	✓
W12	Sprint 2: Core Features	◀ NOW
W13	Sprint 3: Advanced Features	
W14	Sprint 4: Polish & Testing	
W15	Sprint 5: Final Preparation	
W16	UAS: Final Demo Day	

🎯 Sprint 2 Goal: Implement main screens, navigation, dan data layer untuk project

OUTLINE PERTEMUAN

01

Sprint 1 Review

Checkpoint dan feedback

02

UI Implementation

Building screens dengan Compose

03

Navigation Setup

NavHost, routes, dan arguments

04

Data Layer

Repository pattern dan data sources

05

State Management

ViewModel dan UI State

06

Sprint 2 Tasks

Work session dan deliverables

BAGIAN 1

SPRINT 1 REVIEW

Checkpoint dan Feedback

SPRINT 1 CHECKLIST

Pastikan tim sudah menyelesaikan:

- ☐ GitHub repository dengan proper structure
- ☐ Clean Architecture folders (data/, domain/, presentation/)
- ☐ Koin DI setup dengan modules
- ☐ GitHub Actions CI passing
- ☐ README.md lengkap (team, features, tech stack)
- ☐ Project plan document

 **Jika ada yang belum selesai, kerjakan paralel dengan Sprint 2!**

COMMON ISSUES & SOLUTIONS

✗ CI failing

✓ Check Gradle version, ensure test classes exist

✗ Koin not resolving

✓ Verify all modules are loaded in startKoin{}

✗ Git conflicts

✓ Pull before push, communicate with team

✗ Project not building

✓ Sync Gradle, invalidate caches, check JDK version

BAGIAN 2

UI IMPLEMENTATION

Building Screens dengan Compose

SCREEN PLANNING

Typical app screens (sesuaikan dengan project kalian):

Splash Screen	App loading, branding	Optional
Home/List Screen	Main content display, list/grid items	Required
Detail Screen	Item details, actions (edit/delete)	Required
Add/Edit Screen	Form input untuk create/update	Required
Settings Screen	App preferences, about	Optional
Search/Filter	Find specific items	Bonus

SCREEN FILE STRUCTURE

```
presentation/
├── screens/
│   ├── home/
│   │   ├── HomeScreen.kt      # Main composable
│   │   ├── HomeViewModel.kt  # Screen logic
│   │   ├── HomeUiState.kt    # UI state sealed class
│   │   └── components/      # Screen-specific components
│   │       ├── ItemCard.kt
│   │       └── EmptyState.kt
│   ├── detail/
│   │   ├── DetailScreen.kt
│   │   ├── DetailViewModel.kt
│   │   └── DetailUiState.kt
│   ├── add/
│   │   ├── AddEditScreen.kt
│   │   ├── AddEditViewModel.kt
│   │   └── AddEditUiState.kt
│   └── settings/
│       └── SettingsScreen.kt
// Shared components go in:
presentation/components/
├── LoadingIndicator.kt
├── ErrorMessage.kt
└── CommonButton.kt
```

COMPOSE SCREEN TEMPLATE

```
@Composable
fun HomeScreen(
    viewModel: HomeViewModel = koinViewModel(),
    onNavigateToDetail: (Long) -> Unit,
    onNavigateToAdd: () -> Unit
) {
    val uiState by viewModel.uiState.collectAsState()

    Scaffold(
        topBar = {
            TopAppBar(title = { Text("My App") })
        },
        floatingActionButton = {
            FloatingActionButton(onClick = onNavigateToAdd) {
                Icon(Icons.Default.Add, "Add")
            }
        }
    ) { padding ->
        when (val state = uiState) {
            is HomeUiState.Loading -> LoadingIndicator()
            is HomeUiState.Success -> {
                ItemList(
                    items = state.items,
                    onItemClick = onNavigateToDetail,
                    modifier = Modifier.padding(padding)
                )
            }
            is HomeUiState.Error -> ErrorMessage(state.message)
            is HomeUiState.Empty -> EmptyState()
        }
    }
}
```

REUSABLE COMPONENTS

LoadingIndicator.kt

```
@Composable
fun LoadingIndicator(
    modifier: Modifier = Modifier
) {
    Box(
        modifier = modifier.fillMaxSize(),
        contentAlignment = Alignment.Center
    ) {
        CircularProgressIndicator()
    }
}
```

ErrorMessage.kt

```
@Composable
fun ErrorMessage(
    message: String,
    onRetry: (() -> Unit)? = null,
    modifier: Modifier = Modifier
) {
    Column(
        modifier = modifier.fillMaxSize(),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
        Text(message, color = MaterialTheme.colorScheme.error)
        onRetry?.let {
            Button(onClick = it) { Text("Retry") }
        }
    }
}
```

ItemCard.kt

```
@Composable
fun ItemCard(item: Item, onClick: () -> Unit, modifier: Modifier = Modifier) {
    Card(modifier = modifier.fillMaxWidth().clickable(onClick = onClick)) {
        Column(modifier = Modifier.padding(16.dp)) {
            Text(item.title, style = MaterialTheme.typography.titleMedium)
            Text(item.subtitle, style = MaterialTheme.typography.bodySmall)
        }
    }
}
```

BAGIAN 3

NAVIGATION SETUP

NavHost, Routes, dan Arguments

DEFINING ROUTES

navigation/Routes.kt

```
// Sealed class for type-safe navigation
sealed class Screen(val route: String) {
    data object Home : Screen("home")
    data object Add : Screen("add")
    data object Settings : Screen("settings")

    // Routes with arguments
    data object Detail : Screen("detail/{itemId}") {
        fun createRoute(itemId: Long) = "detail/$itemId"
    }

    data object Edit : Screen("edit/{itemId}") {
        fun createRoute(itemId: Long) = "edit/$itemId"
    }
}

// Argument keys
object NavArgs {
    const val ITEM_ID = "itemId"
}

// Usage:
// navController.navigate(Screen.Detail.createRoute(123))
// navController.navigate(Screen.Home.route)
```

NAVHOST SETUP

navigation/AppNavHost.kt

```
@Composable
fun AppNavHost(
    navController: NavHostController = rememberNavController(),
    startDestination: String = Screen.Home.route
) {
    NavHost(navController = navController, startDestination = startDestination) {
        composable(Screen.Home.route) {
            HomeScreen(
                onNavigateToDetail = { id -> navController.navigate(Screen.Detail.createRoute(id)) },
                onNavigateToAdd = { navController.navigate(Screen.Add.route) }
            )
        }

        composable(
            route = Screen.Detail.route,
            arguments = listOf(navArgument(NavArgs.ITEM_ID) { type = NavType.LongType })
        ) { backStackEntry ->
            val itemId = backStackEntry.arguments?.getLong(NavArgs.ITEM_ID) ?: return@composable
            DetailScreen(
                itemId = itemId,
                onNavigateBack = { navController.popBackStack() },
                onNavigateToEdit = { navController.navigate(Screen.Edit.createRoute(itemId)) }
            )
        }

        composable(Screen.Add.route) {
            AddEditScreen(onNavigateBack = { navController.popBackStack() })
        }
    }
}
```

NAVIGATION BEST PRACTICES



Use sealed class for routes

Type-safe, refactor-friendly



Pass lambdas, not NavController

Screen tidak depend on navigation



Single NavHost di root

Avoid nested navigation graphs untuk simple apps



Handle back navigation

Gunakan popBackStack() atau navigateUp()



Don't pass complex objects

Pass ID saja, fetch di destination screen



Avoid deep nesting

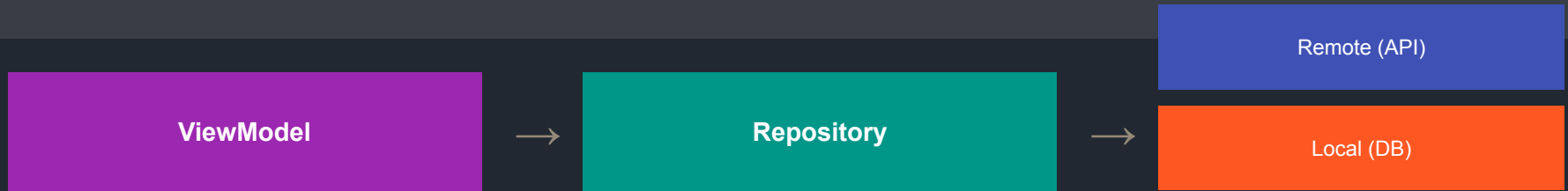
Keep navigation graph flat dan simple

BAGIAN 4

DATA LAYER

Repository Pattern dan Data Sources

DATA LAYER OVERVIEW



Data Layer Structure:

```
data/
├── local/
│   ├── database/
│   │   ├── AppDatabase.kt    # SQLDelight database
│   │   └── ItemQueries.sq    # SQL queries
│   └── datastore/
│       └── PreferencesManager.kt # DataStore preferences
├── remote/
│   ├── api/
│   │   └── ApiService.kt     # Ktor API calls
│   └── dto/
│       └── ItemDto.kt        # Data transfer objects
├── repository/
│   └── ItemRepositoryImpl.kt  # Repository implementation
└── model/
    └── ItemEntity.kt         # Database entities
```

REPOSITORY INTERFACE (DOMAIN)

domain/repository/ItemRepository.kt

```
interface ItemRepository {  
    // Read operations  
    fun getAllItems(): Flow<List<Item>>  
    fun getItemById(id: Long): Flow<Item?>  
    fun searchItems(query: String): Flow<List<Item>>  
  
    // Write operations  
    suspend fun insertItem(item: Item): Long  
    suspend fun updateItem(item: Item)  
    suspend fun deleteItem(id: Long)  
  
    // Sync (if using remote)  
    suspend fun refreshItems()  
}
```



Key Points:

- Interface di domain layer - tidak depend on implementation details
- Flow untuk reactive data - UI auto-update saat data berubah
- Suspend functions untuk write operations - one-shot operations
- Domain models (Item), bukan entities/DTOs

REPOSITORY IMPLEMENTATION

data/repository/ItemRepositoryImpl.kt

```
class ItemRepositoryImpl(
    private val database: AppDatabase,
    private val apiService: ApiService? = null // Optional for offline-only
) : ItemRepository {

    private val queries = database.itemQueries

    override fun getAllItems(): Flow<List<Item>> {
        return queries.selectAll()
            .asFlow()
            .mapToList(Dispatchers.IO)
            .map { entities -> entities.map { it.toDomain() } }
    }

    override fun getItemById(id: Long): Flow<Item?> {
        return queries.selectById(id)
            .asFlow()
            .mapToOneOrNull(Dispatchers.IO)
            .map { it?.toDomain() }
    }

    override suspend fun insertItem(item: Item): Long {
        return withContext(Dispatchers.IO) {
            queries.insert(item.title, item.description, item.createdAt)
            queries.lastInsertRowId().executeAsOne()
        }
    }

    override suspend fun deleteItem(id: Long) {
        withContext(Dispatchers.IO) {
            queries.deleteById(id)
        }
    }
}
```

DOMAIN MODELS & MAPPERS

domain/model/Item.kt

```
// Domain model - UI uses this
data class Item(
    val id: Long = 0,
    val title: String,
    val description: String,
    val createdAt: Long = Clock.System
        .now()
        .toEpochMilliseconds(),
    val isCompleted: Boolean = false
)
```

data/model/Mappers.kt

```
// Entity -> Domain
fun ItemEntity.toDomain() = Item(
    id = id,
    title = title,
    description = description,
    createdAt = createdAt,
    isCompleted = isCompleted == 1L
)

// Domain -> Entity (for insert/update)
fun Item.toEntity() = ItemEntity(
    id = id,
    title = title,
    description = description,
    createdAt = createdAt,
    isCompleted = if (isCompleted) 1L else 0L
)
```

⚠ Separation of Concerns:

- Domain model: Digunakan oleh UI/ViewModel, clean dan readable
- Entity: Database representation, bisa berbeda dari domain
- DTO: API response, di-map ke domain sebelum digunakan

BAGIAN 5

STATE MANAGEMENT

ViewModel dan UI State

UI STATE PATTERN

screens/home/HomeUiState.kt

```
sealed interface HomeUiState {  
    data object Loading : HomeUiState  
  
    data class Success(  
        val items: List<Item>,  
        val searchQuery: String = "",  
        val isRefreshing: Boolean = false  
    ) : HomeUiState  
  
    data class Error(  
        val message: String,  
        val canRetry: Boolean = true  
    ) : HomeUiState  
  
    data object Empty : HomeUiState  
}
```



Benefits of Sealed Interface:

- Exhaustive when - compiler memaksa handle semua state
- Type-safe - tidak perlu null checks
- Clear states - setiap state punya data yang relevan saja

VIEWMODEL IMPLEMENTATION

```
class HomeViewModel(
    private val repository: ItemRepository
) : ViewModel() {

    private val _uiState = MutableStateFlow<HomeUiState>(HomeUiState.Loading)
    val uiState: StateFlow<HomeUiState> = _uiState.asStateFlow()

    init { loadItems() }

    private fun loadItems() {
        viewModelScope.launch {
            repository.getAllItems()
                .catch { e -> _uiState.value = HomeUiState.Error(e.message ?: "Unknown error") }
                .collect { items ->
                    _uiState.value = if (items.isEmpty()) {
                        HomeUiState.Empty
                    } else {
                        HomeUiState.Success(items)
                    }
                }
        }
    }

    fun deleteItem(id: Long) {
        viewModelScope.launch {
            try {
                repository.deleteItem(id)
            } catch (e: Exception) {
                // Handle error - show snackbar via SharedFlow
            }
        }
    }

    fun retry() { loadItems() }
}
```

KOIN DI MODULES

di/Modules.kt

```
val dataModule = module {
    // Database
    single { createDatabase(get()) } // Platform-specific factory

    // Repository
    single<ItemRepository> { ItemRepositoryImpl(get()) }
}

val viewModelModule = module {
    viewModel { HomeViewModel(get()) }
    viewModel { params -> DetailViewModel(get(), params.get()) } // With parameter
    viewModel { AddEditViewModel(get()) }
}

// Combine all modules
val appModules = listOf(dataModule, viewModelModule)

// Initialize in App.kt
fun initKoin() {
    startKoin {
        modules(appModules)
    }
}

// Usage in Composable
@Composable
fun HomeScreen() {
    val viewModel: HomeViewModel = koinViewModel()
    // ...
}
```

BAGIAN 6

SPRINT 2

TASKS

Work Session dan Deliverables

SPRINT 2 GOALS

P0

Main Screens

Home, Detail, Add/Edit screens dengan proper UI

P0

Navigation

Working navigation between all screens

P0

Data Layer

Repository dengan local storage (SQLDelight/DataStore)

P0

CRUD Operations

Create, Read, Update, Delete functionality

P1

State Management

Proper UI states (Loading, Success, Error, Empty)

P1

Basic Styling

Material 3 theming, consistent look

P0 = Must have this sprint | P1 = Should have if time permits

TASK BREAKDOWN CONTOH

Distribute tasks among team members:

Member A	HomeScreen, HomeViewModel, Navigation setup
Member B	DetailScreen, AddEditScreen, Forms validation
Member C	Data layer, Repository, Database setup
All	Code review, testing, bug fixes

Tips for Task Distribution:

- Use GitHub Issues untuk tracking tasks
- Create feature branches per task: feature/home-screen
- Review each other's PRs before merging



WORK SESSION

Waktu: Sisa pertemuan (~60 menit)

1	10 min	Review Sprint 1 deliverables, identify gaps
2	10 min	Plan Sprint 2 tasks, assign to members
3	30 min	Start coding - screens, navigation, atau data layer
4	10 min	Commit & push, create PRs if ready

Target hari ini:

- Semua member punya task yang jelas
- Minimal 1 screen skeleton sudah di-push
- Navigation routes defined

DELIVERABLES SPRINT 2

Bobot: 5% | Deadline: Sebelum Pertemuan 13



Deliverables:

- ☐ Minimal 3 working screens (Home, Detail, Add/Edit)
- ☐ All features accessible dari app (no dead ends)
- ☐ Navigation between screens dengan arguments
- ☐ Data layer dengan Repository pattern
- ☐ Local storage (SQLDelight atau DataStore)
- ☐ Basic CRUD operations working
- ☐ UI States (Loading, Success, Error) implemented



Submission:

- GitHub repo dengan semua code di branch main/develop
- Video demo (1 menit): navigate through all screens, show CRUD
- CI tetap passing

RUBRIK PENILAIAN SPRINT 2

Komponen	Bobot	Kriteria
UI Screens	25%	3+ screens, proper layouts, Material 3
Navigation	20%	Working nav, arguments passing, back handling
Data Layer	25%	Repository, local storage, proper architecture
CRUD Operations	20%	Create, Read, Update, Delete working
Code Quality	10%	Clean code, proper structure, CI passing
★ Bonus: API integration (+10%)		⚠ Late: -10%/day

PREVIEW: SPRINT 3 (MINGGU DEPAN)

Sprint 3: Advanced Features

- **API Integration**

REST API dengan Ktor Client (jika applicable)

- **Search & Filter**

Search functionality, filter options

- **Offline Support**

Cache data, sync strategies

- **Additional Screens**

Settings, Profile, atau feature screens lainnya

🔥 **Persiapan: Complete Sprint 2, review Week 6-7 materials (Networking, Storage)**

TIPS SPRINT 2



Focus on P0 first

Don't over-engineer. Basic functionality > fancy features



Test on device/emulator often

Jangan menumpuk code tanpa running



Small commits

Commit setiap feature kecil selesai, easier to debug



Communicate blockers

Stuck? Ask team atau dosen, jangan diam



Document as you go

Update README dengan setup instructions



Write basic tests

At least verify CRUD works dengan simple tests

COMMON UI PATTERNS

Pull to Refresh

```
PullToRefreshBox(  
    isRefreshing = uiState.isRefreshing,  
    onRefresh = { viewModel.refresh() }  
) {  
    LazyColumn { /* content */ }  
}
```

Confirm Delete

```
var showDialog by remember { mutableStateOf(false) }  
  
if (showDialog) {  
    AlertDialog(  
        onDismissRequest = { showDialog = false },  
        confirmButton = {  
            TextButton(onClick = { viewModel.delete(id) }) {  
                Text("Delete")  
            }  
        },  
        title = { Text("Delete item?") }  
    )  
}
```

Search Bar

```
OutlinedTextField(  
    value = searchQuery,  
    onValueChange = { viewModel.onSearchQueryChange(it) },  
    placeholder = { Text("Search...") },  
    leadingIcon = { Icon(Icons.Default.Search, null) },  
    trailingIcon = {  
        if (searchQuery.isNotEmpty()) {  
            IconButton(onClick = { viewModel.onSearchQueryChange("") }) {  
                Icon(Icons.Default.Clear, "Clear")  
            }  
        }  
    }  
)
```

RESOURCES

Docs

Compose Navigation

developer.android.com/jetpack/compose/navigation

Docs

SQLDelight

cashapp.github.io/sqldelight

Guide

Material 3 Components

m3.material.io/components

Sample

Now in Android

github.com/android/nowinandroid

Sample

KMP Samples

github.com/JetBrains/compose-multiplatform



SESI TANYA JAWAB

Ada pertanyaan tentang Sprint 2?

CHECKLIST SEBELUM PULANG

- ☐ Sprint 1 issues resolved (jika ada)
- ☐ Sprint 2 tasks assigned ke setiap member
- ☐ Navigation routes defined
- ☐ At least 1 screen skeleton created
- ☐ Data layer structure planned
- ☐ Code pushed ke repository
- ☐ Paham deadline dan deliverables Sprint 2



Let's build those core features!

Keep Building!

Sprint 2: Core Features

*"First, solve the problem.
Then, write the code."*

- John Johnson

See you next week for Sprint 3! 

Terima Kasih

Happy Coding! 🚀